E-JAVA LAB ASSIGNMENT-3

20/02/2026

---

1. **Write a Java program that demonstrates basic inheritance by modelling an employee salary system. Create a base class Employee that stores the employee's name and base salary and includes a method to display employee details. Then create a derived class Manager that inherits from Employee and includes an additional bonus amount. The program should calculate and display the manager's total salary by adding base salary and bonus. Create appropriate objects and demonstrate functionality in the main method.**

```
// Base class
class Employee {

    protected String name;
    protected double baseSalary;

    // Constructor
    public Employee(String name, double baseSalary) {
        this.name = name;
        this.baseSalary = baseSalary;
    }

    // Method to display employee details
    public void displayDetails() {
        System.out.println("Employee Name : " + name);
        System.out.println("Base Salary   : ₹" + baseSalary);
    }
```

```
}

// Derived class
class Manager extends Employee {

    private double bonus;

    // Constructor
    public Manager(String name, double baseSalary, double bonus) {
        super(name, baseSalary);   // calling parent constructor
        this.bonus = bonus;
    }

    // Method to calculate total salary
    public double calculateTotalSalary() {
        return baseSalary + bonus;
    }

    // Method to display manager details
    public void displayManagerDetails() {
        displayDetails();   // calling parent method
        System.out.println("Bonus         : ₹" + bonus);
        System.out.println("Total Salary  : ₹" + calculateTotalSalary());
    }
}

// Main class
public class EmployeeSalarySystem {
    public static void main(String[] args) {

        // Creating Manager object
        Manager m1 = new Manager("Rahul", 50000, 10000);

        m1.displayManagerDetails();
```

```
    }
}
```

Output:

Employee Name : Rahul

Base Salary   : ₹50000.0

Bonus   : ₹10000.0

Total Salary : ₹60000.0

2. **Develop a Java program to demonstrate the use of the super keyword. Create a base class Vehicle that stores and displays the base price of a vehicle. Then create a derived class Car that defines its own price but also accesses and displays the parent class price using super. Demonstrate how super is used to refer to parent class variables or methods.**

```
// Base class
class Vehicle {

    double price = 500000;   // base price

    // Method to display vehicle price
    public void displayPrice() {
        System.out.println("Vehicle Base Price : ₹" + price);
    }
}

// Derived class
class Car extends Vehicle {

    double price = 800000;   // car's own price

    // Method to display both prices
```

```
    public void displayDetails() {

        System.out.println("Car Price         : ₹" + price);

        // Accessing parent class variable using super
        System.out.println("Parent Price      : ₹" + super.price);

        // Calling parent class method using super
        super.displayPrice();
    }
}

// Main class
public class VehicleTest {
    public static void main(String[] args) {

        Car c1 = new Car();
        c1.displayDetails();
    }
}
```

Output:

```
Car Price         : ₹800000.0
Parent Price      : ₹500000.0
Vehicle Base Price : ₹500000.0
```

3. **Write a Java program that implements multilevel inheritance to represent a university academic structure. Create a class Person that stores name, a class Student that inherits from Person and stores roll number, and a class GraduateStudent that inherits from Student and stores area**

of specialization. Display complete student details from the final derived class.

```
// Base class
class Person {

    protected String name;

    // Constructor
    public Person(String name) {
        this.name = name;
    }
}

// Derived class (Level 1)
class Student extends Person {

    protected int rollNumber;

    // Constructor
    public Student(String name, int rollNumber) {
        super(name);   // calling Person constructor
        this.rollNumber = rollNumber;
    }
}

// Derived class (Level 2)
class GraduateStudent extends Student {

    private String specialization;

    // Constructor
    public GraduateStudent(String name, int rollNumber, String specialization) {
        super(name, rollNumber);   // calling Student constructor
```

```
        this.specialization = specialization;
    }

    // Method to display complete details
    public void displayDetails() {
        System.out.println("Name           : " + name);
        System.out.println("Roll Number    : " + rollNumber);
        System.out.println("Specialization : " + specialization);
    }
}

// Main class
public class UniversitySystem {
    public static void main(String[] args) {

        // Creating object of final derived class
        GraduateStudent g1 = new GraduateStudent("Rahul", 101, "Computer Science");

        g1.displayDetails();
    }
}
```

Output:

```
Name           : Rahul
Roll Number    : 101
Specialization : Computer Science
```

4. **Create a Java program to demonstrate method overriding. Define a base class Shape with a method area() that displays a general message. Create derived classes such as Circle and Rectangle that override the area() method to compute and**

display their respective areas. Use appropriate formulas and demonstrate overriding through object creation.

```
// Base class
class Shape {

    // Method to be overridden
    public void area() {
        System.out.println("Area calculation for shape.");
    }
}

// Derived class - Circle
class Circle extends Shape {

    double radius;

    public Circle(double radius) {
        this.radius = radius;
    }

    // Overriding method
    @Override
    public void area() {
        double result = Math.PI * radius * radius;
        System.out.println("Area of Circle: " + result);
    }
}

// Derived class - Rectangle
class Rectangle extends Shape {

    double length, width;
```

```
    public Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }

    // Overriding method
    @Override
    public void area() {
        double result = length * width;
        System.out.println("Area of Rectangle: " + result);
    }
}

// Main class
public class ShapeTest {
    public static void main(String[] args) {

        // Base class reference with different objects
        Shape s;

        s = new Circle(7);
        s.area();   // calls Circle's method

        s = new Rectangle(5, 4);
        s.area();   // calls Rectangle's method
    }
}
```

Output:

Area of Circle: 153.93804002589985

Area of Rectangle: 20.0

5. **Write a Java program that demonstrates runtime polymorphism using dynamic method dispatch. Create a base class Payment with a method pay(). Derive subclasses representing different payment methods (e.g., UPI and Card). Use a parent class reference to call the appropriate payment method at runtime and demonstrate dynamic binding.**

```
// Base class
class Payment {

    // Method to be overridden
    public void pay() {
        System.out.println("Processing payment...");
    }
}

// Derived class - UPI
class UPI extends Payment {

    @Override
    public void pay() {
        System.out.println("Payment done using UPI.");
    }
}

// Derived class - Card
class Card extends Payment {

    @Override
    public void pay() {
        System.out.println("Payment done using Card.");
```

```
    }
}

// Main class
public class PaymentSystem {
    public static void main(String[] args) {

        Payment p;   // parent class reference

        p = new UPI();   // object of UPI
        p.pay();         // calls UPI method

        p = new Card();  // object of Card
        p.pay();         // calls Card method
    }
}
```

Output:

Payment done using UPI.

Payment done using Card.

6. **Develop a Java program using an abstract class to model a banking system. Create an abstract class Bank containing an abstract method to return the rate of interest. Implement this method in subclasses representing different banks. Demonstrate abstraction by accessing interest rates using a base class reference.**

```
// Abstract class
abstract class Bank {

    // Abstract method
    abstract double getRateOfInterest();
```

```
}

// Subclass - SBI
class SBI extends Bank {

    @Override
    double getRateOfInterest() {
        return 6.5;
    }
}

// Subclass - HDFC
class HDFC extends Bank {

    @Override
    double getRateOfInterest() {
        return 7.2;
    }
}

// Main class
public class BankingSystem {
    public static void main(String[] args) {

        Bank b;   // reference of abstract class

        b = new SBI();
        System.out.println("SBI Interest Rate: " + b.getRateOfInterest() + "%");

        b = new HDFC();
        System.out.println("HDFC Interest Rate: " + b.getRateOfInterest() + "%");
    }
}
```

Output:

SBI Interest Rate: 6.5%

HDFC Interest Rate: 7.2%

7. **Write a Java program demonstrating the use of the final keyword in inheritance. Create a class containing a final method that displays policy rules. Derive another class from it and show that the final method is inherited but cannot be overridden. Explain the significance of using final methods.**

```
// Base class
class Policy {

    // Final method (cannot be overridden)
    public final void showRules() {
        System.out.println("Policy Rules:");
        System.out.println("1. Follow company guidelines.");
        System.out.println("2. Maintain discipline.");
    }
}

// Derived class
class InsurancePolicy extends Policy {

    // ❌ This will cause error if uncommented
    /*
    public void showRules() {
        System.out.println("Overridden Rules");
    }
    */

    // Additional method
    public void displayPolicy() {
```

```
        System.out.println("Insurance Policy Details Displayed.");
    }
}

// Main class
public class FinalKeywordDemo {
    public static void main(String[] args) {

        InsurancePolicy obj = new InsurancePolicy();

        obj.showRules();      // inherited final method
        obj.displayPolicy();
    }
}
```

Output:

Policy Rules:

1. Follow company guidelines.
2. Maintain discipline.

Insurance Policy Details Displayed.

8. **Design a Java program to demonstrate aggregation. Create a class Employee and another class Department that contains an object of Employee. Display department details along with employee information. Show how aggregation represents a "has-a" relationship between classes.**

```
// Employee class
class Employee {

    private int empId;
    private String empName;

    // Constructor
```

```
    public Employee(int empId, String empName) {
        this.empId = empId;
        this.empName = empName;
    }

    // Method to display employee details
    public void displayEmployee() {
        System.out.println("Employee ID   : " + empId);
        System.out.println("Employee Name : " + empName);
    }
}

// Department class (Aggregation)
class Department {

    private int deptId;
    private String deptName;
    private Employee employee;   // has-a relationship

    // Constructor
    public Department(int deptId, String deptName, Employee employee) {
        this.deptId = deptId;
        this.deptName = deptName;
        this.employee = employee;
    }

    // Method to display department details
    public void displayDepartment() {
        System.out.println("Department ID   : " + deptId);
        System.out.println("Department Name : " + deptName);

        // Accessing Employee object
        employee.displayEmployee();
    }
```

```
}

// Main class
public class AggregationDemo {
    public static void main(String[] args) {

        // Creating Employee object
        Employee e1 = new Employee(101, "Rahul");

        // Passing Employee object to Department (Aggregation)
        Department d1 = new Department(10, "IT", e1);

        d1.displayDepartment();
    }
}
```

Output:

```
Department ID   : 10
Department Name : IT
Employee ID   : 101
Employee Name : Rahul
```

9. **Create a user-defined package containing a class with a protected data member. Create another class in a different package that accesses the protected member through inheritance. Demonstrate how access protection works across packages.**

Step 1: Create Package pack1 with Base Class

```
package pack1;
```

```
public class Person {

  // Protected data member
  protected String name = "Rahul";

  // Method to display name
  protected void display() {
     System.out.println("Name : " + name);
  }
}
```

✅ Step 2: Create Package pack2 with Derived Class

```
package pack2;

import pack1.Person;

// Inheriting from Person
public class Student extends Person {

  public void showDetails() {

     // Accessing protected members through inheritance
     System.out.println("Accessing from subclass:");
     System.out.println("Name : " + name);   // accessible
     display();                              // accessible
  }
}
```

✅ Step 3: Main Class (in same package pack2)

```
package pack2;
```

```
public class TestAccess {
  public static void main(String[] args) {

    Student s = new Student();
    s.showDetails();
  }
}
```

Output:

```
Accessing from subclass:
Name : Rahul
Name : Rahul
```

10. **Create a Java interface that declares methods to start and stop a device. Implement this interface in a class representing a specific device such as a fan or motor. Demonstrate interface implementation and method invocation using interface reference variables.**

```
// Interface
interface Device {

  // Abstract methods
  void start();
  void stop();
}

// Implementing class
class Fan implements Device {

  @Override
```

```
    public void start() {
        System.out.println("Fan started.");
    }

    @Override
    public void stop() {
        System.out.println("Fan stopped.");
    }
}

// Main class
public class InterfaceDemo {
    public static void main(String[] args) {

        // Interface reference
        Device d;

        d = new Fan();   // object of implementing class

        d.start();       // calling start()
        d.stop();        // calling stop()
    }
}
```

Output:

Fan started.

Fan stopped.